

aucmat: matrix-first ROC analysis for high-dimensional biomarker screening in R

Lucas VHH Tran^{1,*}

¹CK-CARE, Davos, Switzerland

*lucas.tran@ck-care.ch

Abstract

Summary: The area under the receiver operating characteristic curve (AUC) is the standard metric for evaluating continuous biomarkers against binary outcomes. When screening thousands of molecular biomarkers, researchers need matrix-wide AUC estimation with proper inference, multiplicity adjustment, and stability assessment. We present **aucmat**, an R package providing a matrix-first architecture: `aucmat(X, y)` computes direction-preserving AUCs with DeLong or stratified-bootstrap confidence intervals for every column of a numeric matrix in a single call. The package supports Benjamini-Hochberg multiplicity adjustment, three hypothesis-testing frameworks (superiority, non-inferiority, equivalence via TOST), an omnibus Wald test for equality of multiple correlated AUCs via the joint DeLong covariance matrix (testing, e.g., whether several candidate biomarkers have statistically indistinguishable AUCs before selecting a panel), and bootstrap rank-stability analysis. A two-stage Hurdle-AUC model handles highly zero-inflated omics data. Eight simulation generators produce correlated biomarkers with controlled properties under exchangeable, AR(1), block, or user-supplied correlation structures with built-in calibration diagnostics. Over a dozen visualization functions are provided. The package is freely available under the MIT license at <https://github.com/vanhungtran/aucmat>.

Availability: **aucmat** v0.2.0, R $\geq$ 4.0.0, MIT license. <https://github.com/vanhungtran/aucmat> (source), <https://vanhungtran.github.io/aucmat> (documentation).

Contact: lucas.tran@ck-care.ch

1 Introduction

Receiver operating characteristic (ROC) analysis is the dominant framework for evaluating continuous biomarkers against binary outcomes in diagnostic medicine, prognostic modeling, and molecular screen-

ing [Hanley and McNeil, 1982, DeLong et al., 1988, Pepe, 2004]. The area under the ROC curve (AUC) summarises the probability that a randomly chosen positive subject has a higher biomarker value than a randomly chosen negative subject.

Modern molecular profiling technologies routinely measure thousands of features on a few hundred subjects. A proteomics panel may contain 5,000 proteins; a transcriptomics experiment 20,000 genes. The core task – identifying which biomarkers discriminate cases from controls – requires computing and testing an AUC for each feature, adjusting for multiplicity, comparing top candidates, and assessing ranking stability. Yet no single widely-used R package integrates these steps.

The R ecosystem provides excellent point solutions. **pROC** [Robin et al., 2011] is the gold standard: it computes DeLong confidence intervals using the Sun & Xu (2014) $\mathcal{O}(N \log N)$ algorithm [Sun and Xu, 2014], tests paired or unpaired AUC differences via `roc.test()`, provides partial AUC with bootstrap inference, binormal and density-smoothed ROC curves, and multiclass AUC via Hand & Till [Hand and Till, 2001]. However, **pROC** operates one biomarker at a time. **ROCR** [Sing et al., 2005] offers 28 performance measures and visualisation but no statistical inference. **precRec** [Saito and Rehmsmeier, 2017] adds fast C++-based precision-recall curves and confidence bands but provides no DeLong inference or multiplicity adjustment. **colAUC** [Tuszynski and Dietze, 2024] computes column-wise AUCs efficiently but provides no inference whatsoever. **dtComb** [Yerlitas et al., 2025] combines two biomarkers via 142 methods but does not perform matrix-wide screening. **cancerclass** [Budczies et al., 2024] builds classifiers from high-dimensional data but lacks DeLong-based screening.

aucmat fills this gap with a *matrix-first* design. The primary function `aucmat(X, y)` accepts $\mathbf{X} \in \mathbb{R}^{n \times p}$ and a binary outcome $\mathbf{y} \in \{0, 1\}^n$, returning point estimates, standard errors, confidence intervals, raw and multiplicity-adjusted p -values, and diagnostic meta-data for every biomarker in one call. Downstream

functions for comparison, stability, cross-validation, panel construction, and power analysis consume the same S3 object, preserving outcome encoding throughout.

2 Methods

2.1 Direction-preserving AUC

For biomarker X and binary outcome $Y \in \{0, 1\}$, the AUC equals the Mann-Whitney U probability:

$$\text{AUC} = P(X_i > X_j \mid Y_i = 1, Y_j = 0) \quad (1)$$

Most software reports $\max(\text{AUC}, 1 - \text{AUC})$, silently reversing the biomarker. `aucmat` preserves the observed direction, reporting AUC_{raw} (may be < 0.5), $\text{AUC}_{\text{strength}} = 0.5 + |\text{AUC}_{\text{raw}} - 0.5|$ (for ranking), and `effect_direction`. A tumour suppressor with $\text{AUC}_{\text{raw}} = 0.15$ discriminates as strongly as an oncogene with 0.85.

2.2 DeLong variance and joint covariance

The DeLong estimator [DeLong et al., 1988, Sun and Xu, 2014] uses placement values:

$$V_{10}(X_i) = \frac{1}{n_0} \sum_{j: Y_j=0} [\mathbf{1}(X_j < X_i) + \tfrac{1}{2}\mathbf{1}(X_j = X_i)], \quad (2)$$

$$V_{01}(X_j) = \frac{1}{n_1} \sum_{i: Y_i=1} [\mathbf{1}(X_i > X_j) + \tfrac{1}{2}\mathbf{1}(X_i = X_j)] \quad (3)$$

where $i \in \mathcal{I}_1$ and $j \in \mathcal{I}_0$. Let S_{10} and S_{01} be the sample variances of the placement values $V_{10}(X_i)$ and $V_{01}(X_j)$, respectively. The DeLong variance estimator is:

$$\widehat{\text{Var}}(\widehat{\text{AUC}}) = \frac{S_{10}}{n_1} + \frac{S_{01}}{n_0} \quad (4)$$

For p biomarkers on common subjects, the $p \times p$ joint covariance is:

$$\widehat{\Sigma} = \frac{1}{n_1} \widehat{\text{Cov}}(\mathbf{V}_{10}) + \frac{1}{n_0} \widehat{\text{Cov}}(\mathbf{V}_{01}) \quad (5)$$

This supports an omnibus Wald test of $H_0 : \text{AUC}_1 = \dots = \text{AUC}_p$. With contrast matrix $\mathbf{C}$ (default: comparing biomarkers 2, ..., p against biomarker 1):

$$W = (\mathbf{C}\hat{\boldsymbol{\theta}})^\top (\mathbf{C}\widehat{\Sigma}\mathbf{C}^\top)^{-1} (\mathbf{C}\hat{\boldsymbol{\theta}}) \sim \chi_r^2 \quad (6)$$

using the effective rank r via eigenvalue tolerance. At $p = 2$, W equals the squared paired-DeLong z -statistic within numerical tolerance.

2.3 Multiplicity and stability

Screening p biomarkers produces p tests. `aucmat` applies Benjamini-Hochberg [Benjamini and Hochberg, 1995] (default), Holm, or Bonferroni correction via `stats::p.adjust()`, reporting adjusted p -values as q -values. For TOST equivalence [Schuirmann, 1987], the classic two one-sided tests procedure yields $p_{\text{equiv}} = \max(p_{\text{lower}}, p_{\text{upper}})$; equivalence is declared when $p_{\text{equiv}} < \alpha$ after multiplicity adjustment on this combined value. The default multiplicity method is BH for `aucmat()` screening and none for `compare_auc()` (users should specify `adjust = "BH"` for pairwise comparisons).

Bootstrap rank-stability (`auc_stability()`) resamples data B times (stratified by outcome), recomputes the full screening, and reports median/IQR rank distributions, top- k selection probabilities, and pairwise co-selection frequencies [Efron and Tibshirani, 1994].

2.4 Hurdle-AUC for zero-inflated data

Standard AUC assumes continuous biomarkers. On zero-inflated data (scRNA-seq, microbiome), 60%+ of values are zero, and standard AUC degrades to near 0.5. The Hurdle-AUC models zeros and expression as separate processes:

$$P(X_j = 0 \mid Y) = \text{logistic}(\beta_0 + \beta_1 Y) \quad (7)$$

Stage 2 (magnitude): standard AUC on $\{X_j \mid X_j > 0\}$. The composite score for observation i is:

$$S_i = \begin{cases} \hat{p}_i & \text{if } X_{ij} = 0 \\ 0.5\hat{p}_i + 0.5\tilde{x}_{ij} & \text{if } X_{ij} > 0 \end{cases} \quad (8)$$

where $\hat{p}_i$ is the predicted non-zero probability from Stage 1 and $\tilde{x}_{ij}$ is the normalised biomarker value.

2.5 Multivariate simulation engine

The simulation engine is a core contribution of `aucmat`. We describe it in full mathematical detail, as it underpins the package's reproducibility guarantees and distinguishes it from ad-hoc simulation approaches.

2.5.1 The equal-variance binormal model

These generators enable power and sample-size exploration, method benchmarking under controlled correlation structures, and fully reproducible vignettes. Under the standard binormal model [Hanley and McNeil, 1982], for biomarker $k \in \{1, \dots, p\}$:

$$X_k \mid Y = 0 \sim \mathcal{N}(\mu_{k0}, \sigma^2), \quad X_k \mid Y = 1 \sim \mathcal{N}(\mu_{k1}, \sigma^2) \quad (9)$$

The variance σ^2 is shared across classes and, unless otherwise specified, across biomarkers. Post-generation standardisation to zero mean and unit variance is an affine transform that preserves both AUC and Pearson correlation. The standardised mean separation $\delta_k = (\mu_{k1} - \mu_{k0})/\sigma$ determines the AUC via:

$$\text{AUC}_k = \Phi\left(\frac{\delta_k}{\sqrt{2}}\right) \quad (10)$$

where $\Phi(\cdot)$ is the standard normal CDF. Inverting gives the closed-form calibration used by all `aucmat` simulators:

$$\delta_k = \sqrt{2} \cdot \Phi^{-1}(\text{AUC}_k) \quad (11)$$

For $\text{AUC} = 0.8$, $\delta \approx 1.19$; for $\text{AUC} = 0.9$, $\delta \approx 1.81$. The class means are centred so the n -weighted grand mean is zero:

$$\mu_{k0} = \delta_k \cdot \left(-\frac{n_1}{n}\right), \quad \mu_{k1} = \delta_k \cdot \left(\frac{n_0}{n}\right) \quad (12)$$

2.5.2 Class-conditional multivariate normal (`simulate_auc_matrix`)

For p biomarkers with correlation matrix $\mathbf{R}$, the class-conditional distribution generalises Eq. (9) to the multivariate case:

$$\mathbf{X} \mid Y = 0 \sim \mathcal{N}_p(\boldsymbol{\mu}_0, \mathbf{R}), \quad \mathbf{X} \mid Y = 1 \sim \mathcal{N}_p(\boldsymbol{\mu}_1, \mathbf{R}) \quad (13)$$

where $\boldsymbol{\mu}_0 = (\mu_{10}, \dots, \mu_{p0})^\top$ and $\boldsymbol{\mu}_1 = (\mu_{11}, \dots, \mu_{p1})^\top$ follow Eq. (12). Both classes share the same $\mathbf{R}$.

The outcome Y is fixed *before* any biomarker is drawn. In `outcome_mode = "exact"`, $n_1 = \lfloor n\pi \rfloor$ observations are assigned $Y = 1$ with randomised order; in `"bernoulli"`, each $Y_i \sim \text{Bernoulli}(\pi)$. When Y is user-supplied, its exact values and row order are preserved.

The matrix $\mathbf{R}$ is built from a named structure:

- **exchangeable**: $R_{ij} = \rho$ for all $i \neq j$. Requires $\rho > -1/(p-1)$ for positive definiteness.
- **AR(1)**: $R_{ij} = \rho^{|i-j|}$.
- **block**: $R_{ij} = \rho_w$ if i, j in same block, ρ_b otherwise. Block sizes must sum to p .
- **user**: R_{ij} directly from the supplied matrix.

When $\mathbf{R}$ is not positive definite, `feasibility = "nearest"` projects to the closest valid correlation matrix via eigenvalue clipping followed by re-normalisation to unit diagonal. The adjustment is reported in `$feasibility` with Frobenius norm and maximum element-wise change.

After generation, each biomarker is standardised to zero mean and unit variance (an affine transform that preserves both AUC and Pearson correlation). Achieved correlation is computed on within-class centred residuals to avoid mixture-induced inflation (Simpson's paradox).

2.5.3 Gaussian copula with perturbation (`simulate_auc_copula`)

This simulator separates correlation control from discriminative signal:

Phase 1 – Gaussian Copula. For each class independently, n_c draws are taken from $\mathcal{N}_{p+1}(\mathbf{0}, \mathbf{R}_{\text{work}})$, where $\mathbf{R}_{\text{work}}$ is a $(p+1) \times (p+1)$ matrix combining the biomarker correlation $\mathbf{R}$ (upper-left $p \times p$) with latent outcome correlations $\boldsymbol{\rho}$ (last row/column). The latent correlations are approximated from target AUCs:

$$\rho_k^{\text{latent}} = \frac{\delta_k}{\sqrt{1 + \delta_k^2}} \quad (14)$$

Each column is rank-transformed to uniform quantiles, then probit-transformed to standard normal scores. This copula step preserves rank correlations exactly regardless of marginal distributions, making `simulate_auc_copula` the recommended choice when non-Gaussian marginals are desired or when decoupling AUC control from correlation structure is critical (e.g., for stress-testing methods under distributional misspecification).

Phase 2 – Iterative Perturbation. After Phase 1, each biomarker's empirical AUC, $\widehat{\text{AUC}}_k$, is compared to its target. The gap $\Delta_k = \text{AUC}_k - \widehat{\text{AUC}}_k$ is translated to a mean shift via the binormal calibration:

$$\Delta\delta_k = \sqrt{2} \left[\Phi^{-1}(\text{AUC}_k) - \Phi^{-1}(\widehat{\text{AUC}}_k) \right] \quad (15)$$

The shift $\Delta\delta_k \cdot \sigma_k \cdot \eta^t$ is applied class-conditionally (positive class receives $+(n_0/n)$ times the shift, negative class $-(n_1/n)$), where $\eta \in (0, 1)$ is the step decay and t the iteration index. Perturbation stops when $|\Delta_k| < \tau$ for all k or after T_{max} iterations.

2.5.4 Hurdle simulator (`simulate_hurdle_auc`)

Generates zero-inflated biomarkers via a two-stage process. For biomarker j :

Stage 1 – Zero hurdle. Logistic parameters are solved from user-specified per-class zero rates:

$$\beta_0 = \text{logit}(p_0), \quad p_0 = P(X_j = 0 \mid Y = 0) \quad (16)$$

$$\beta_1 = \text{logit}(p_1) - \text{logit}(p_0), \quad p_1 = P(X_j = 0 \mid Y = 1) \quad (17)$$

For each subject, $Z_{ij} \sim \text{Bernoulli}(\text{logistic}(\beta_0 + \beta_1 Y_i))$ determines whether $X_{ij} = 0$.

Stage 2 – Magnitude. For observations with $Z_{ij} = 0$ (expressed), values are drawn from a log-normal distribution with class-conditional means calibrated to achieve the target non-zero AUC via Eq. (11):

$$X_{ij} \mid X_{ij} > 0, Y_i = c \sim \text{LogNormal}(\mu_c, \sigma^2) \quad (18)$$

where $\mu_1 - \mu_0$ is set by the binormal shift. Values are clipped to $[0, \infty)$. The log-normal magnitude model is a pragmatic choice for positive-valued omics measurements; users may apply monotone transforms to approximate alternative marginal shapes if needed. The Hurdle-AUC reflects the joint discriminative ability of the probability of being expressed and the magnitude among expressed observations, rather than treating zeros as tied minimal values as standard AUC does. Users should inspect both components (hurdle and magnitude) via `plot_hurdle_diagnostics()` whenever Hurdle-AUC and standard AUC differ substantially.

2.5.5 Other simulators

`generate_data_probit()` uses a latent probit formulation where $\mathbf{X}$ and a latent $Y^* \sim \mathcal{N}(0, 1)$ are jointly $\mathcal{N}_{p+1}(\mathbf{0}, \Sigma)$, with $Y = \mathbf{1}\{Y^* > \Phi^{-1}(1 - \pi)\}$. The full $(p + 1) \times (p + 1)$ correlation matrix simultaneously controls both biomarker-biomarker and biomarker-outcome associations. The latent correlations $\rho_k = \text{Cor}(X_k, Y^*)$ are calibrated numerically via bisection over a large calibration sample ($n_{\text{cal}} = 100,000$).

`generate_data_analytical()` constructs biomarkers sequentially from highest to lowest AUC. For $k \geq 2$, X_k is built as a linear combination of $X_1, \dots, X_{k-1}$ (weighted by $\mathbf{R}_{k,1:k-1} \mathbf{R}_{1:k-1,1:k-1}^{-1}$ to achieve target correlations) plus an independent residual carrying the remaining mean separation. This is fast (closed-form, no calibration loop) but links AUC and correlation.

`generate_auc_vector()` constructs a single score with exact empirical AUC via rank assignment. Given n_1 positives and n_0 negatives, achievable AUCs form a discrete grid with step $\Delta_{\text{AUC}} = 1/(n_1 n_0)$. The required rank sum is solved and normal quantile scores are assigned.

`generate_auc_cor_vector()` extends this with a strictly increasing Box-Cox transform tuned to achieve a target Pearson correlation without changing the AUC (since monotone transforms preserve ranks).

`simulate_auc_correlation()` repeatedly draws from the binormal model and records achieved (AUC, r) pairs to characterise their sampling distribution.

2.5.6 Simulation validation

`validate_simulation()` repeats a specification across R independent replicates with deterministic seed streams (seed + $r - 1$). For each replicate it

records achieved AUCs and correlations, then reports:

$$\text{Bias}_k = \frac{1}{R} \sum_{r=1}^R (\widehat{\text{AUC}}_{kr} - \text{AUC}_k) \quad (19)$$

$$\text{RMSE}_k = \sqrt{\frac{1}{R} \sum_{r=1}^R (\widehat{\text{AUC}}_{kr} - \text{AUC}_k)^2} \quad (20)$$

$$\text{MCSE}_k = \frac{\text{SD}_k}{\sqrt{R}} \quad (21)$$

plus target-interval hit rates for both AUCs and correlations. A bias large relative to MCSE signals a systematic calibration error.

3 Implementation

`aucmat` is implemented in pure R (6,200 lines, 34 source files). The primary function performs six steps: (1) input validation, (2) outcome normalisation (accepting logical, 0/1, factor, or character), (3) matrix-wide AUC via Mann-Whitney rank formulation, (4) DeLong or bootstrap inference, (5) multiplicity adjustment, (6) assembly with ranking and warnings.

All functions return S3 objects with `print()`, `summary()`, `plot()`, and `as.data.frame()` methods. Structured error classes allow programmatic handling. An RNG-safety mechanism (`with_seed()`) restores `.Random.seed` on exit.

Dependencies: pROC (≥ 1.18), mvtnorm, ggplot2, rlang. Optional Suggests: GGally, glmnet, matrixStats, knitr, rmarkdown, ggrepel.

4 Usage

```
fit <- aucmat(X, y, ci = "delong", adjust = "BH")
print(fit); summary(fit)
plot_auc_rank(fit, n_label = 4)
```

```
compare_auc(fit, X, y, reference = "X1")
compare_auc(fit, X, y, reference = "X1",
  hypothesis = "noninferiority", margin = 0.05)
compare_auc(fit, X, y, reference = "X1",
  hypothesis = "equivalence", margin = 0.15)
compare_auc_global(fit, X, y)
roc_test(X[,1], y)
```

```
sim <- simulate_hurdle_auc(n = 400, prevalence = 0.3,
  target_hurdle_aucs = c(0.85, 0.72),
  zero_rate_neg = c(0.55, 0.80),
  zero_rate_pos = c(0.25, 0.70))
fit <- hurdle_auc(as.matrix(sim$data[,1:2]),
  sim$data$truth)
```

```
cv <- cv_aucmat(X, y, n_folds = 5, seed = 42)
panel <- fit_auc_panel(X[,top4], y, method = "ridge")
power_auc_matrix(target_aucs = 0.70,
  power = 0.80, prevalence = 0.3)
```

5 Benchmarking

Table 1: Empirical coverage of DeLong 95% confidence intervals, averaged across 27 simulation conditions (200 replicates each).

AUC	$n = 100$	$n = 300$	$n = 500$
0.65	0.898	0.937	0.957
0.75	0.915	0.943	0.933
0.85	0.883	0.923	0.905

Screening throughput. On a desktop workstation (AMD Ryzen 3.5 GHz, 32 GB RAM, R 4.5.1, Windows 11), `aucmat` screens 100, 1,000, and 10,000 biomarkers ($n = 500$, DeLong inference) in 0.54, 5.02, and 59.76 seconds. The equivalent per-column `pROC` loop requires 0.68, 6.42, and an estimated 64 seconds.

Simulation calibration. We assessed DeLong inference calibration using `simulate_auc_matrix()` with three biomarkers per replicate (equal target AUCs of 0.65, 0.75, or 0.85), exchangeable and AR(1) correlation structures ($\rho = 0, 0.3$), outcome generated in exact mode, and 200 independent replicates per condition. Coverage of the first biomarker’s 95% CI was recorded. Across 27 conditions ($n = 100, 300, 500$; prevalence = 0.2, 0.3, 0.5; AUC = 0.65, 0.75, 0.85; averaged over correlation structures), DeLong coverage ranged from 0.898 to 0.957, converging to 0.95 as n increased (Table 1). The slightly liberal coverage at small n is consistent with the known finite-sample behaviour of the DeLong estimator, motivating the recommendation of bootstrap CIs when $n < 100$ per class. Power exceeded 0.85 for AUC ≥ 0.65 at $n = 100$ and reached 1.0 for AUC ≥ 0.75 at $n \geq 300$.

6 Comparison with existing packages

To our knowledge, no other single R package natively provides all of the following in one integrated workflow without requiring user-side loops or custom scripting: (i) single-call, matrix-wide AUC estimation with integrated DeLong inference and multiplicity adjustment while preserving the observed biomarker direction; (ii) non-inferiority and equivalence (TOST) hypothesis tests for paired AUC comparisons; (iii) an omnibus

Wald test for three or more correlated AUCs via the joint DeLong covariance matrix; (iv) bootstrap rank-stability analysis with top- k selection probabilities and pairwise co-selection frequencies; and (v) a two-stage Hurdle-AUC model for highly zero-inflated omics data that recovers discriminative signal where standard AUC degrades to near chance.

For a single biomarker with smoothed ROC curves, `pROC` remains the standard. For speed on $> 10^5$ columns, `colAUC`’s Wilcoxon-based computation is recommended, though it sacrifices inference for throughput; `aucmat` is optimised for matrix-scale inference up to at least 10^4 biomarkers on commodity hardware, with screening time scaling approximately linearly in p at fixed n . For precision-recall of imbalanced data, `precrec` is preferred. For combining two biomarkers, `dtComb` provides the most exhaustive methods. For end-to-end biomarker screening with full inference, `aucmat` is the recommended tool.

7 Discussion

`aucmat` provides a unified, statistically principled framework for biomarker screening. Its matrix-first design reduces the code burden from dozens of scripting steps to a few function calls.

Limitations. The current version (0.2.0) is restricted to binary outcomes for the primary workflow (multiclass is experimental). DeLong requires common subjects; unpaired comparisons are not implemented (`pROC` provides this). Partial AUC is an internal engine not yet exposed in `aucmat()`. The DeLong estimator is slightly liberal at small n (coverage ≈ 0.90 at $n = 100$), consistent with its finite-sample behaviour; bootstrap intervals are recommended when $n < 100$ per class.

Future work. v0.3: nested cross-validation for penalised panel models. v0.4: multiclass AUC with one-versus-rest, one-versus-one, and Hand-Till estimators. v0.5: cumulative/dynamic time-dependent AUCs for survival outcomes [Heagerty et al., 2000, Blanche et al., 2013]. v0.6: sparse-matrix and on-disk backends for genomics-scale data.

Acknowledgements

We thank The Kühne Foundation for financial support. Xavier Robin and colleagues are acknowledged for `pROC`, which provided the statistical foundation and ROC curve objects. Hadley Wickham’s `ggplot2` and the `mvtnorm` authors (Genz, Bretz, et al.) are gratefully acknowledged. The Sun & Xu (2014) fast

Table 2: Feature comparison of R packages for ROC analysis and biomarker screening. + = natively supported without user-side loops or custom coding; -- = not supported or requires manual implementation. Verified against CRAN documentation as of July 2026.

Feature	aucmat	pROC	precrec	ROCR	colAUC	dtComb
Matrix-first screening	+	—	—	—	+	—
Direction-preserving AUC	+	—	—	—	—	—
DeLong CI	+	+	—	—	—	—
Stratified bootstrap CI	+	+	—	—	—	+
Multiplicity adjustment	+	—	—	—	—	—
One-sided tests	+	+	—	—	—	—
Paired comparisons	+	+	—	—	—	+
Non-inferiority/equivalence	+	—	—	—	—	—
Omnibus Wald (3+ AUCs)	+	—	—	—	—	—
Bootstrap rank stability	+	—	—	—	—	—
Partial AUC	+	+	+	—	—	—
Multivariate simulation	+	—	—	—	—	—
Simulation diagnostics	+	—	—	—	—	—
Cross-validated AUC	+	—	—	—	—	+
Panel (ridge/lasso)	+	—	—	—	—	+
Hurdle-AUC (zero-inflated)	+	—	—	—	—	—
Precision-Recall curves	+	—	+	—	—	—
Power / sample size	+	+	—	—	—	—
Multiclass (Hand-Till)	+	+	—	—	+	—
ROC smoothing	+	+	—	—	—	—
Group-stratified screening	+	—	—	—	—	—

DeLong algorithm implemented in pROC since v1.9 informs **aucmat**’s variance computation.

Funding

This work was supported by The Kühne Foundation.

References

- Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.
- Paul Blanche, Jean-François Dartigues, and Hélène Jacqmin-Gadda. Estimating and comparing time-dependent areas under receiver operating characteristic curves for censored event times with competing risks. *Statistics in Medicine*, 32(30):5381–5397, 2013.
- Jan Budczies, Daniel Kosztyla, Christian von Törne, Albrecht Stenzinger, Silvia Darb-Esfahani, Manfred Dietel, Carsten Denkert, and Frederick Klauschen. cancerclass: Development and validation of diagnostic tests from high-dimensional molecular data, 2024. R package version 1.49.0, Bioconductor.
- Elizabeth R DeLong, David M DeLong, and Daniel L Clarke-Pearson. Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach. *Biometrics*, 44(3):837–845, 1988.
- Bradley Efron and Robert J Tibshirani. *An Introduction to the Bootstrap*. CRC Press, Boca Raton, FL, 1994.
- David J Hand and Robert J Till. A simple generalisation of the area under the ROC curve for multiple class classification problems. *Machine Learning*, 45(2):171–186, 2001.
- James A Hanley and Barbara J McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.
- Patrick J Heagerty, Thomas Lumley, and Margaret S Pepe. Time-dependent ROC curves for censored survival data and a diagnostic marker. *Biometrics*, 56(2):337–344, 2000.
- Margaret S Pepe. *The Statistical Evaluation of Medical Tests for Classification and Prediction*. Oxford University Press, New York, 2004.

- Xavier Robin, Natacha Turck, Alexandre Hainard, Natalia Tiberti, Frédérique Lisacek, Jean-Charles Sanchez, and Markus Müller. pROC: an open-source package for R and S+ to analyze and compare ROC curves. *BMC Bioinformatics*, 12:77, 2011. doi: 10.1186/1471-2105-12-77.
- Takaya Saito and Marc Rehmsmeier. precrec: fast and accurate precision-recall and ROC curve calculations in R. *Bioinformatics*, 33(1):145–147, 2017. doi: 10.1093/bioinformatics/btw570.
- Donald J Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987.
- Tobias Sing, Oliver Sander, Niko Beerenwinkel, and Thomas Lengauer. ROCR: visualizing classifier performance in R. *Bioinformatics*, 21(20):3940–3941, 2005. doi: 10.1093/bioinformatics/bti623.
- Xu Sun and Weichao Xu. Fast implementation of DeLong’s algorithm for comparing the areas under correlated receiver operating characteristic curves. *IEEE Signal Processing Letters*, 21(11):1389–1393, 2014. doi: 10.1109/LSP.2014.2337313.
- Jarek Tuszynski and Michael Dietze. caTools: Tools: moving window statistics, GIF, Base64, ROC AUC, etc., 2024. R package version 1.18.3.
- Serra Ilayda Yerlitas, Serra Bersan Gengec, Necla Kochan, Gozde Erturk Zararsiz, Selcuk Korkmaz, and Gokmen Zararsiz. dtComb: A comprehensive R library and web tool for combining diagnostic tests. *The R Journal*, 17(1):1–20, 2025. doi: 10.32614/RJ-2025-036.